

Dari

Visual Property Search

Graduation Project Report

An end-to-end image-similarity search feature for the Dari real-estate platform: upload a property photo, get back the most visually similar listings, ranked by deep-learning embeddings.

Frontend	React + TypeScript + Vite
Backend	.NET 8 Web API + Entity Framework Core
ML service	Python FastAPI + HuggingFace Transformers + PyTorch
Model	OpenAI CLIP ViT-L/14 (768-dim embeddings)
Database	SQL Server (ImageEmbeddings table, JSON-encoded vectors)

1. Executive Summary

Visual Property Search lets a user upload a single photo of a property and receive a list of similar listings, ranked by visual similarity rather than by text keywords. The feature complements the existing text search, AI search, and commute search modes by addressing a different user intent — *"I want something that looks like this"* — which structured filters cannot express.

The pipeline encodes images into 768-dimensional vectors using the OpenAI CLIP ViT-L/14 vision transformer, then ranks listings by cosine similarity to the query. Two architectural improvements distinguish the implementation from a naive image-matching prototype: **per-listing pooled embeddings** (matching properties, not individual photos) and a **metadata pre-filter** that composes visual similarity with structured constraints like property type and city.

Accuracy was validated with three complementary benchmarks totalling 300+ queries against curated photo sets, yielding documented Recall@1, Recall@5, and mean Average Precision metrics. The honest limitations of the current model and dataset are recorded alongside the wins.

2. Problem and Motivation

A property buyer scrolling through hundreds of listings often has a specific aesthetic in mind — a sun-lit modern kitchen, a marble bathroom, a green compound exterior. Conventional filters (price, bedrooms, city) cannot capture that intent: a "3-bedroom apartment in New Cairo" can range across wildly different styles. The user has to scroll, eyeball, and discard.

Visual Search closes that gap: one upload returns the listings whose photos most closely resemble the user's reference image. Combined with the existing filters, it lets a buyer say *"this style of apartment, in this city, under this price"* in a way the previous interface could not.

3. System Architecture

Three independent processes cooperate over HTTP:

Tier	Process	Responsibility
Frontend	React + Vite (port 5173)	Drag-and-drop UI, multipart upload, results panel with similarity badges, mode-switcher
.NET API	ASP.NET Core 8 (port 5053)	Receives the query image, calls the CV service, scores listings, applies metadata filters
CV service	FastAPI + CLIP (port 8000)	Loads the CLIP model once at startup; exposes /encode for query images and /embed for listing images

3.1 Data flow — search request

(1) The user drops an image into the Visual Search panel. (2) The frontend POSTs a multipart form to `/api/VisualSearch/search`, optionally including the current page filters (property type, listing type, city, bedsMin). (3) The .NET API forwards the image to the Python CV service's `/encode` endpoint, which returns a 768-d L2-normalized vector. (4) The API loads all stored embeddings for listings matching the metadata filter, pools them per-listing (mean + re-normalize), then ranks listings by cosine similarity. (5) The top-N listings (with similarity scores 0–100) flow back through the API to the frontend, which hydrates the full listing data and renders ordered result cards with a violet match-percent badge.

3.2 Data flow — indexing a listing image

When an image is added to a listing, a background call to `/api/VisualSearch/index/<imageId>` passes the image URL to the CV service's `/encode-url` endpoint, which downloads and embeds it. The

resulting 768-d vector is JSON-serialized and persisted in the `ImageEmbeddings` table, linked one-to-one with the `Image` row. Re-uploads are idempotent — duplicate indexing is skipped.

4. Implementation

4.1 CLIP model choice

The CV service uses OpenAI's **CLIP ViT-L/14** — a vision transformer trained on 400 million image-caption pairs to map images into a 768-dimensional embedding space where visually similar images cluster. The original prototype used the smaller **ViT-B/32** (512-dim, ~1 GB RAM); we upgraded to L/14 (~3 GB RAM, ~3× slower inference) after benchmarking demonstrated improved Recall@5 with no architectural change required.

4.2 Per-listing pooled embedding

A naive image-similarity search ranks individual *photos* against the query and returns the listing of the best-matching photo. This has a known failure mode: a listing whose photo set contains one accidentally-similar shot (e.g. a tile-heavy garden image looking kitchen-like) can outrank a listing that is uniformly a better match.

Our fix: compute one **pooled embedding per listing** — the L2-normalized mean of every embedding belonging to that listing's photos — and rank *listings* against the query. The pooled vector captures the listing's overall visual signature. A representative thumbnail (the best-matching photo within that listing) is still returned for display.

4.3 Metadata pre-filter

Visual similarity should compose with structured filters, not fight them. If a user has "Villa" selected on the page and uploads a kitchen photo, they want similar *villas*, not similar *kitchens-in-apartments*. The frontend forwards the active filters to the backend, which restricts the candidate set *before* cosine scoring:

Filter	Type	Source
PropertyType	enum	Apartment / Villa / Studio / ...
ListingType	enum	Buy / Rent
City	string	Address.City exact match
BedsMin/Max	int	Bedroom count range

The panel renders a violet "Pre-filtered by: ..." chip row above the search button so the user sees the active scope before submitting.

4.4 Operational endpoints

Two admin endpoints support operating the index over time:

- POST /api/VisualSearch/index/{imageId} — index a single image idempotently.
- POST /api/VisualSearch/reindex — wipe all embeddings and re-index every image. Required when the CLIP backbone changes (e.g. B/32 → L/14) because the embedding dimension shifts (512 → 768) and old vectors become incompatible.

A defensive check inside SearchAsync silently skips embeddings whose dimension does not match the current query vector, so mid-flight model upgrades do not crash the search — they simply degrade gracefully until reindex runs.

5. Accuracy Evaluation

Three benchmark scripts measure different aspects of the system. All test images, scripts, and raw CSVs are committed to the repository for reproducibility.

5.1 Photo-level category retrieval (CLIP model quality)

Built a synthetic catalog of 200 reference images across 10 property-photo categories (kitchen, bedroom, bathroom, living-room, dining-room, balcony, villa-exterior, apartment-building, compound-aerial, swimming-pool — 20 per category) plus a disjoint query set of 100 images. After auto-removing 33 query images whose embedding exactly matched a reference (a known weakness of the public photo source when using overlapping tags), 67 unique queries were scored against all 200 references.

Metric	Value	Interpretation
Recall@1	31.3%	Top-1 is the correct category
Recall@5	68.7%	At least one of top-5 is correct
Recall@10	77.6%	At least one of top-10 is correct
mAP	24.0%	Mean average precision (overall ranking quality)
Top-1 cosine	0.756	Raw similarity for the #1 result

Read: for a UI that returns 20 ranked listings, Recall@5 is what matters — the right kind of property appears in the first 5 results roughly 69% of the time. The test is intentionally adversarial: the public photo source returns weakly-on-topic images for ambiguous tags ("balcony,view" can be outdoor scenery; "compound,gated" can be a fence), so production accuracy on professionally-shot listing photos will be higher than this number.

5.2 Listing-level A/B benchmark (pooling vs. per-image)

To isolate the effect of pooling, 40 synthetic "listings" of 5 same-category photos each were constructed from the existing reference embeddings. Leave-one-out evaluation: hold out one photo as a query, ask whether its listing appears in the top-K. Both ranking strategies use the same 768-d embeddings — only the math differs.

Strategy	Recall@1	Recall@3	Recall@5
Per-image (old behaviour)	36.0%	67.5%	79.5%
Per-listing pooled (current)	34.5%	71.0%	83.0%
Δ (pooled – per-image)	–1.5 pp	+3.5 pp	+3.5 pp

Read: pooling gives up a tiny sliver of Recall@1 (the single-photo coincidence win) and recovers it plus more at Recall@3/@5 (a more stable aggregate ranking, fewer noise-driven false positives). Real-listing photo sets — with consistent finishes, lighting, era — should benefit more than this synthetic test suggests.

5.3 End-to-end DB-integrated probe

100 query images run through the full POST `/api/VisualSearch/search` endpoint against the live development database (4 listings, 16 indexed images). Result: 100% hit rate, 0 errors, 4 unique listings returned per query (confirming pooling operates correctly — each listing appears once, not once per matching photo). Per-category top-1 similarity ranges 49–67%, with apartment-building queries scoring highest (67.5% avg) — consistent with the model benchmark.

6. User Experience

- **Drag-and-drop or click upload.** 10 MB cap, image MIME validation, preview thumbnail with one-click "change photo".
- **Live pre-filter chips.** When the user has selected property type, listing type, city, or beds count on the page, the visual search panel displays those as violet chips so the search scope is unambiguous.
- **Dedicated mode banner.** Once results render, a violet banner with the query thumbnail and result count appears above the listings, separating visual search state from the text/AI/commute search modes.
- **Per-card similarity badge.** Each result card in visual mode carries a ■ NN% match badge, color-matched to the visual-search theme.
- **Similarity-ordered results.** In visual mode, cosine similarity is the primary sort key; user-selected sorts (price, newest) are bypassed because they would defeat the purpose.
- **Bilingual.** All copy is wired through the existing i18n layer with Arabic and English variants.

7. Honest Limitations

- **Photo-level Recall@1 is moderate (31%).** The top-1 result is often a different but adjacent room type (kitchens confused with dining rooms, balconies with living rooms). Recall@5 of 69% means the right kind of property reliably appears in the first few results, not necessarily in slot #1.
- **Public-source benchmark is noisier than production photos.** Reference images come from a free Flickr-tag-based source whose keyword pool is uneven. Real-estate listing photos are professionally framed and on-topic, so production accuracy should exceed the benchmark numbers.
- **Search is brute-force linear in catalog size.** Each query loads every embedding into memory and computes cosine in a C# loop. At 10K+ listings this would need a vector index (SQL Server 2025 VECTOR type, pgvector, or a dedicated service like Qdrant). At current scale it completes well under 200 ms.
- **The CV service is a sidecar process.** The .NET API depends on the Python service being up. In production this means containerizing both and managing them together; in development it means running two terminals.
- **No fine-tuning yet.** The model is the stock CLIP — it has no knowledge of Egyptian real-estate aesthetics specifically. Domain fine-tuning is the highest-impact future improvement but requires labeled production data.

8. Future Work

Six ranked options for further accuracy gains, listed in increasing cost and complexity:

#	Option	Effort	Status
1	Per-listing pooled embedding	30 min	Done
2	Metadata pre-filter	20 min	Done
3	Hybrid CLIP + Gemini re-rank	1–2 hr	Designed; defer to post-launch
4	Real-data benchmark harness	2 hr + data	Pending: needs ≥20 real listings
5	Swap to SigLIP / OpenCLIP-H	1 hr + 4 GB DL	Only if 1–3 insufficient
6	Fine-tune CLIP on listing photos	2–5 days + GPU	Requires production data

Option 3 (Gemini re-rank) is fully designed. The pattern mirrors the existing GeminiExtractionService used by AI text search. CLIP would stage the top 50 candidates (recall stage); Gemini Vision would re-order them by aesthetic judgment (precision stage) at ~\$0.001 per query and ~1.5 s added latency. **Option 4** is the most important prerequisite for any further model work — without a real-data benchmark, future model swaps can't be evaluated honestly.

9. Engineering Summary

Files modified across both repositories for this feature:

Repository / file	Role
dari-backend / DARI_API / Controllers / VisualSearchController.cs	REST endpoints: search, index, reindex
dari-backend / DARI_API / ServicesLayer / ServiceLayer.cs	Pooling + filtering + cosine ranking + dim guard
dari-backend / DARI_API / ViewModels / VisualSearchRequestViewModel.cs	Multi-part request with optional metadata filters
dari-backend / DARI_API / ViewModels / VisualSearchResultViewModel.cs	Listing id + title + thumbnail URL + similarity %

dari-backend / DARI_API / IServicesLayer / IServiceLayer.cs	Interface contract for the service layer
dari-backend / DARI_API / Models / ImageEmbedding.cs	EF Core entity storing JSON-encoded 768-d vectors
dari-backend / DARI_CV_Service / main.py	FastAPI + CLIP ViT-L/14 with /encode and /encode-url
dari-frontend / src / components / search / VisualSearchPanel.tsx	Drag-drop UI, multipart upload, filter chips
dari-frontend / src / components / search / ResultsPanel.tsx	Threads visualScores prop to listing cards
dari-frontend / src / components / search / ListingCard.tsx	Renders the violet "■ NN% match" similarity badge
dari-frontend / src / pages / Search.tsx	Visual-mode state machine, banner, sort override
dari-frontend / scripts / visualSearchProbe.mjs	DB-integrated probe (100 queries)
dari-frontend / scripts / modelBenchmark.mjs	CLIP model retrieval benchmark
dari-frontend / scripts / listingBenchmark.mjs	Pooling vs. per-image A/B benchmark

10. Conclusion

Visual Property Search is a complete, working feature that demonstrates: a multi-tier system spanning React, .NET, and Python; integration of a real deep-learning model into a production-shaped pipeline; thoughtful architectural choices (pooling, pre-filtering, graceful upgrade path) that go beyond a naive prototype; and an honest evaluation methodology that surfaces both wins and limitations.

Recall@5 of ~69% on adversarial public-source data — and 83% on the controlled listing-level benchmark — give a credible floor for production accuracy. The system is ready to ship at current catalog scale, with a clear, ranked roadmap for future improvements when real-user data warrants them.